

RISCV workflow

The program begins in the main function inside of sh.c:

```
// Ensure that three file descriptors are open.
while((fd = open("console", O_RDWR)) >= 0){
    if(fd >= 3){
        close(fd);
        break;
    }
}

// Read and run input commands.
while(getcmd(buf, sizeof(buf)) >= 0){
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
        cmd++;
    if (*cmd == '\0') // blank command
        continue;

    add_history(cmd); // Save to history
```

The program attempts to open a console if its already not open. After that it attempts to read input commands through getcmd(). Otherwise if a command is entered, fork() is called and a child is created to run the command.

```
} else {
    if(fork() == 0)
        runcmd(parsecmd(cmd));
    wait(0);
}
}
```

getcmd() works the following way (updated to accommodate the history command):

```
int getcmd(char *buf, int nbuf) {
    printf("$ ");
    int i = 0;
    reset_history_index();

    while (1) {
        char c;
        if (read(0, &c, 1) < 1)
            return -1;
    }
}
```

It prints the dollar sign and reads the buffer. Checks are made to make sure if it is a blank command, 'cd' command or any other command which can either be invalid or an actual syscall of xv6.

After that runcmd() uses a switch case to choose which type of command was executed:

```
switch(cmd->type){
default:
    panic("runcmd");

case EXEC:
    ecmd = (struct execcmd*)cmd;
    if(ecmd->argv[0] == 0)
        exit(1);
    exec(ecmd->argv[0], ecmd->argv);
    fprintf(2, "exec %s failed\n", ecmd->argv[0]);
    break;

case REDIR:
    rcmd = (struct redircmd*)cmd;
    close(rcmd->fd);
    if(open(rcmd->file, rcmd->mode) < 0){
        fprintf(2, "open %s failed\n", rcmd->file);
        exit(1);
    }
    runcmd(rcmd->cmd);
}
```

Runcmd contains different types of commands:

Type	Meaning	Struct used	Example Command
EXEC	Execute a program	struct execcmd	ls, echo hello
REDIR	Input/output redirection	struct redircmd	ls > out.txt
PIPE	Pipe between two commands	struct pipecmd	`ls
LIST	Sequential commands (;)	struct listcmd	ls; echo done
BACK	Background execution (&)	struct backcmd	sleep 10 &

In our case the question asks for the execution of the program, so runcmd calls exec()

Exec():

```

    exit(1);
    exec(ecmd->argv[0], ecmd->argv);
    fprintf(f, "exec %s failed\n", ecmd->argv[0]);
}

```

exec looks up the executable file on the disk. Loads the program into memory, sets up user stack arguments and starts the execution of the file. It is to be noted that exec is a system call and therefore requires assistance from the kernel.

